

Tracing Struktur Data

1. Trie — Insert & Search

Data yang diinsert:

- "stasiun surabaya gubeng" → S01
- "stasiun surabaya pasar turi" → S02
- "halte diponegoro" → B08

Semua nama diubah ke lowercase sebelum diproses.

Insert "stasiun surabaya gubeng"

Setiap karakter dibuat node baru jika belum ada di `children`. Setelah traversal selesai di karakter terakhir `g`, node tersebut di-set `isEndOfWord = true` dan referensi ke objek Node S01 ditambahkan ke `associatedNodes`.

Search prefix "sta"

- Traversal: `root → s → t → a`
- Node `a` berhasil ditemukan.
- `findAllNodes` dijalankan secara rekursif ke seluruh subtree di bawahnya.
- Semua node yang memiliki `isEndOfWord = true` dikumpulkan dan dikembalikan.
- Hasil: **S01 dan S02**

Search prefix "xyz"

- Traversal: `root → x` menghasilkan `null` karena `x` tidak ada di `children` root.
- Fungsi langsung mengembalikan list kosong tanpa traversal lebih lanjut.

2. MinHeap — Insert & ExtractMin

Skenario: insert Gubeng (20), Wonokromo (5), Gedangan (12)

Step 1 — Insert Gubeng (20)

- a. Array: [Gubeng:20]
- b. current = 0, tidak ada parent → selesai.

Step 2 — Insert Wonokromo (5)

- a. Array sementara: [Gubeng:20, Wonokromo:5]
- b. current = 1, parent(1) = 0
- c. heap[1].value (5) < heap[0].value (20) → **swap**
- d. Array: [Wonokromo:5, Gubeng:20]

Step 3 — Insert Gedangan (12)

- a. Array sementara: [Wonokromo:5, Gubeng:20, Gedangan:12]
- b. current = 2, parent(2) = 0
- c. heap[2].value (12) > heap[0].value (5) → tidak swap
- d. Array: [Wonokromo:5, Gubeng:20, Gedangan:12]

ExtractMin #1

- a. min = Wonokromo:5
- b. Pindah elemen terakhir ke root: [Gedangan:12, Gubeng:20]
- c. minHeapify(0): left(0) = Gubeng:20, right(0) = tidak ada
- d. smallest = 0 (12 < 20) → tidak swap
- e. Output: **Wonokromo (5)**

ExtractMin #2

- a. min = Gedangan:12
- b. Pindah elemen terakhir ke root: [Gubeng:20]
- c. minHeapify(0): tidak ada child
- d. Output: **Gedangan (12)**

ExtractMin #3

- a. min = Gubeng:20
- b. Array: []
- c. Output: **Gubeng (20)**

Urutan output: Wonokromo (5) → Gedangan (12) → Gubeng (20)

3. BFS — Minimum Transit

Skenario A — Ada edge langsung

Asal: S01 (Stasiun Surabaya Gubeng) | Tujuan: B06 (Halte Tunjungan)

Edge keluar dari S01: E01→S03, E11→S02, E52→B20, E64→B06

Inisialisasi

a. queue: [[S01]]

b. visited: {S01}

Iterasi 1

a. Poll [S01]

b. Periksa tetangga S01: S03, S02, B20 belum dikunjungi → masuk queue

c. B06 == tujuan → **DITEMUKAN**

Path final	S01 → B06
Jumlah transit	1
Total waktu	12 menit (E64)
Total biaya	Rp 5.000 (E64)

BFS selesai di iterasi pertama karena ada edge langsung dari S01 ke B06.

Skenario B — Tidak ada edge langsung

Asal: B13 (Halte ITS Sukolilo) | Tujuan: B07 (Halte Urip Sumoharjo)

Edge keluar dari B13: E45→B14, E56→B18, E62→S01

Inisialisasi

a. queue: [[B13]]

b. visited: {B13}

Iterasi 1

a. Poll [B13]

b. Tambah B14, B18, S01 ke queue

Iterasi 2

- a. Poll [B13, B14]
- b. Tetangga B14: tambah B19

Iterasi 3

- a. Poll [B13, B18]
- b. Tetangga B18: tambah B17

Iterasi 4

- a. Poll [B13, S01]
- b. Tetangga S01: tambah S03, S02, B20, B06 (B13 visited)

Iterasi 5

- a. Poll [B13, B14, B19]
- b. Tetangga B19: B14 & B20 visited → tidak ada node baru

Iterasi 6

- a. Poll [B13, B18, B17]
- b. Tetangga B17: tambah B11

... iterasi berlanjut ...

Iterasi N

- a. Poll [B13, S01, B06]
- b. Tetangga B06: ditemukan B07 == tujuan → **DITEMUKAN**

Path final	B13 → S01 → B06 → B07
Jumlah transit	3
Total waktu	18 + 12 + 8 = 38 menit
Total biaya	Rp 6.000 + Rp 5.000 + Rp 3.000 = Rp 14.000

BFS menjamin ini adalah path dengan transit paling sedikit dari B13 ke B07.

4. Dijkstra — Rute Tercepat

Skenario: S03 (Stasiun Wonokromo) → B06 (Halte Tunjungan), mode waktu

Edge relevan:

Edge	Rute	Waktu
E02	S03 → S01	8 menit
E03	S03 → S04	12 menit
E15	S03 → S05	10 menit
E22	S03 → B01	10 menit
E11	S01 → S02	15 menit
E52	S01 → B20	10 menit
E64	S01 → B06	12 menit

Inisialisasi

- a. jarak[semua] = MAX, jarak[S03] = 0
- b. PQ: [(S03, 0)]
- c. visited: {}

Step 1 — Poll S03 (jarak=0)

- a. visited: {S03}
- b. Update tetangga: S01=8, S04=12, S05=10, B01=10
- c. PQ: [(S01, 8), (S05, 10), (B01, 10), (S04, 12)]

Step 2 — Poll S01 (jarak=8)

- a. visited: {S03, S01}
- b. S03: $8+8=16 > 0 \rightarrow \text{skip}$
- c. Update: S02=23, B20=18, B06=20, B13=26
- d. PQ: [(S05, 10), (B01, 10), (S04, 12), (B20, 18), (B06, 20), (S02, 23), (B13, 26)]

Step 3 — Poll S05 (jarak=10)

Update: S06 = $10+8 = 18$

Step 4 — Poll B01 (jarak=10)

Update: $B11 = 10 + 12 = 22$

Step 5 — Poll S04 (jarak=12)

Update: $S08 = 12 + 7 = 19$

... iterasi berlanjut ...

Step N — Poll B06 (jarak=20)

a. B06 == tujuan → **SELESAI**

b. Rekonstruksi: $B06 \leftarrow \text{sebelum}[B06]=S01 \leftarrow \text{sebelum}[S01]=S03$

c. Balik urutan: $[S03, S01, B06]$

Path final	$S03 \rightarrow S01 \rightarrow B06$
Rute	Stasiun Wonokromo → Stasiun Surabaya Gubeng → Halte Tunjungan
Total waktu	20 menit